TOPSTECHNOLOGIES
Training | Outsourcing | Placement | Study Abroad

ASSESSMENT FILE
SOFTWARE ENGINEERING
ASSESSMENT CODE : M3-A1
M3 - Introduction to Programming

SECTION A — CONCEPT APPLICATION

1. **SCENARIO**

   You are a junior developer at a fintech startup building a digital wallet app. The team is debating between Waterfall and Agile methodology. The project has frequently changing requirements and the client expects to review working features every two weeks.

   Question: Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

➔ For this project, I would recommend using the **Agile development methodology** because the project requirements change frequently and the client wants to review working features every two weeks. Agile divides the project into small iterations (sprints), allowing developers to make changes quickly based on client feedback. This helps deliver useful features continuously while reducing the risk of building the wrong product.

If the team chose the **Waterfall model** instead, they could face the following risks:

1. Difficulty handling changing requirements: Waterfall requires most requirements to be fixed before development begins. Frequent changes would force costly redesign and redevelopment.

2. Late customer feedback: Since testing and customer review happen near the end of the project, the client may discover problems only after significant development has already been completed, increasing both cost and time.

2. **SCENARIO**

   You are collaborating on a college project website with three teammates. Just before a client demo, one teammate pushed incomplete code directly to the main branch, breaking the layout for everyone else on the team.

   Question: Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

➔ The team should follow a **Git Feature Branch Workflow**.
Instead of working directly on the main branch, each developer should create a separate feature branch for their assigned task. After completing and testing the work, the developer should submit a Pull Request so teammates can review the code before merging it into the main branch. This prevents unfinished or broken code from affecting everyone.

Useful Git commands include:

1. **git checkout -b feature-name**
   Creates and switches to a new feature branch where development is done safely.
2. **git add .**
   Adds modified files to the staging area before committing.
3. **git commit -m "message"**
   Saves changes with a meaningful description.
4. **git push origin feature-name**
   Uploads the feature branch to the remote repository for review.
5. **git merge feature-name**
   Merges the reviewed feature branch into the main branch after approval.

This workflow reduces the chance of broken code reaching the main branch and improves collaboration.

3. SCENARIO

You are building an online internship application form for a college placement portal. The form must collect: applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

Question: List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

➔ The appropriate HTML input types are:

Field	**HTML Input Type**
Applicant Name	text
Email Address	email
Phone Number	tel
Preferred Domain	select
Resume Upload	file

To enforce the required constraints without JavaScript:

- Use the **required** attribute on every field so the form cannot be submitted if any field is empty.
- Use **type="email"** for the email field so the browser checks whether the entered value follows a valid email format.

Example:

```
<input type="text" required>
<input type="email" required>
<input type="tel" required>

<select required>
    <option value="">Choose Domain</option>
    <option>Web</option>
    <option>Data</option>
    <option>Mobile</option>
    <option>AI</option>
```

</select>

<input type="file" required>

However, HTML5 validation alone is **not sufficient** for production because users can disable browser validation or send requests directly to the server. Therefore, server-side validation is necessary to verify all submitted data securely.

4. **SCENARIO**

You are designing a news portal that displays articles in a 3-column grid on desktop screens and in a single-column layout on mobile devices. Your team wants to use Bootstrap's grid system to achieve this without writing any custom media query CSS.

Question: Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

➔ Bootstrap uses a **12-column responsive grid system**. Rows are divided into 12 equal columns, and developers assign column widths using predefined classes for different screen sizes.

For this layout, each article should occupy:

- **4 columns** on medium and larger screens (12 ÷ 4 = 3 columns)
- **12 columns** on small screens (full width)

Class:

Explanation:

- **col-12**
  Makes the article occupy the full row on small/mobile screens, creating a single-column layout.
- **col-md-4**
  On medium screens and larger, each article occupies 4 of the 12 columns, resulting in three columns per row.

Bootstrap automatically switches layouts at different screen sizes without requiring custom media queries.

5. **SCENARIO**

   You are writing a C program to record daily rainfall data for a month (30 days). You need to store all 30 readings, calculate the monthly average, and then identify which specific days recorded rainfall above that average.

   Question: Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

➔ An **array** is the correct data structure because it stores multiple values of the same type under a single variable name. Instead of creating 30 separate variables for each day's rainfall, an array allows the program to access all values using indexes, making the code shorter, easier to manage, and suitable for loops.

The program should follow these two steps:

1. **Calculate the average**
   - Read all 30 rainfall values.
   - Store them in the array.
   - Add them together.
   - Divide the total by 30 to find the average.
2. **Find above-average days**
   - Traverse the array again.
   - Compare each day's rainfall with the calculated average.
   - Print the day numbers where rainfall is greater than the average.

These steps cannot be completed in a single loop because the average is not known until all 30 values have been entered and the total has been calculated.

6. SCENARIO

You are debugging a C program that uses a pointer to traverse a string entered by the user. The program works correctly when the user types a name, but crashes immediately when the input field is left empty (the user just presses Enter).

Question: Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

➔ The program crashes because when the user presses **Enter** without typing any characters, the string is empty. If the program dereferences a pointer without checking whether it points to the null terminator **('\0')**, it may access invalid memory, leading to a crash.

Before dereferencing the pointer, the program should check:

```
if (*ptr != '\0')
```

or ensure that the pointer is not at the end of the string before accessing its value.

Pointer traversal moves through memory by incrementing the pointer:

```
while (*ptr != '\0')
{
        ptr++;
}
```

Array indexing accesses characters using positions:

```
while(str[i] != '\0')
{
   i++;
}
```

Pointer traversal works directly with memory addresses, whereas array indexing uses numeric indexes. Array indexing often makes empty-string conditions easier to notice because checking **str[0] == '\0'** clearly indicates that the input is empty.

SECTION D — AI-Augmented Learning

1. **The exact prompt(s) my gave the AI tool.**

➔ Write a C program that:

1. Accepts exactly 10 integers from the user using a loop and stores them in an array.
2. Finds and displays the maximum value, minimum value, and arithmetic mean (displayed as a float with 2 decimal places).
3. Sorts the array in ascending order using any sorting method and displays the sorted array.
4. Prints whether the mean is closer to the minimum value, closer to the maximum value, or exactly midway between them.

The program should compile with GCC and use standard C.

2. **The AI's original code and my corrected version.**

➔ **AI's Original Code (original.c)**

```c
#include <stdio.h>

int main()
{
    int arr[10];
    int i, j;
    int min, max;
    int sum = 0;
    float mean;

    printf("Enter 10 integers:\n");

    for(i = 0; i < 10; i++)
    {
        scanf("%d", &arr[i]);
    }

    min = arr[0];
    max = arr[0];

    for(i = 0; i < 10; i++)
    {
        if(arr[i] < min)
```

```
            min = arr[i];

        if(arr[i] > max)
            max = arr[i];

        sum += arr[i];
    }

    mean = sum / 10.0;

    for(i = 0; i < 9; i++)
    {
        for(j = i + 1; j < 10; j++)
        {
            if(arr[i] > arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    printf("\nMinimum = %d\n", min);
    printf("Maximum = %d\n", max);
    printf("Mean = %.2f\n", mean);

    printf("\nSorted Array:\n");

    for(i = 0; i < 10; i++)
        printf("%d ", arr[i]);

    printf("\n");

    if((mean - min) < (max - mean))
        printf("Mean is closer to Minimum.\n");
    else if((mean - min) > (max - mean))
        printf("Mean is closer to Maximum.\n");
    else
        printf("Mean is exactly midway.\n");

    return 0;
}
```

Corrected Version (corrected.c)

```
#include <stdio.h>
#include <stdlib.h>

int main()
```

```c
{
    int arr[10];
    int i, j;
    int min, max;
    int sum = 0;
    float mean;
    float distMin, distMax;

    printf("Enter 10 integers:\n");

    for(i = 0; i < 10; i++)
    {
        scanf("%d", &arr[i]);
    }

    min = arr[0];
    max = arr[0];

    for(i = 0; i < 10; i++)
    {
        if(arr[i] < min)
            min = arr[i];

        if(arr[i] > max)
            max = arr[i];

        sum += arr[i];
    }

    mean = sum / 10.0;

    for(i = 0; i < 9; i++)
    {
        for(j = i + 1; j < 10; j++)
        {
            if(arr[i] > arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    printf("\nMinimum = %d\n", min);
    printf("Maximum = %d\n", max);
    printf("Mean = %.2f\n", mean);

    printf("\nSorted Array:\n");
```

```
    for(i = 0; i < 10; i++)
        printf("%d ", arr[i]);

    printf("\n");

    distMin = mean - min;
    if(distMin < 0)
        distMin = -distMin;

    distMax = max - mean;
    if(distMax < 0)
        distMax = -distMax;

    if(distMin < distMax)
        printf("Mean is closer to Minimum.\n");
    else if(distMin > distMax)
        printf("Mean is closer to Maximum.\n");
    else
        printf("Mean is exactly midway.\n");

    return 0;
}
```

3. **A 3–4 line note explaining what you changed and why the AI's original version needed the correction.**

➔ I tested the program with identical values, positive and negative integers, and values where the mean lies between the minimum and maximum. I found that the original program compared the distances directly, which could give incorrect results when negative values were involved.

I corrected the program by calculating the absolute distance between the mean and the minimum/maximum values before comparing them.

```
float distMin, distMax;
```

```
distMin = mean - min;
    if(distMin < 0)
        distMin = -distMin;

    distMax = max - mean;
    if(distMax < 0)
        distMax = -distMax;
```

This makes the program produce the correct result for all valid integer inputs.